

- **Weakness 1: Inconsistency in the testing methodology**

In section 5.2 the strategy “bottom-up and thread testing” is declared, while in section 5.3 the actual testing phase follows a feature-driven approach. The test plan uses an iterative structure in which at each cycle one feature is tested together with all the components required for that feature, before moving to the next one. On the other hand, the bottom-up approach would require testing individual components first, such as the DBMS or the API Gateway, and only after that is tested the functioning of the higher-level components that depend on them. This misalignment creates a problem for the unambiguity target quality, raising doubts about the actual testing strategy employed.

- **Weakness 2: Critical components excluded from testing**

Some important components, such as the DBMS, API Gateway and the front-end, are excluded from the testing phase without any sufficient reason. The DBMS, for example, is assumed to be functional, leaving a critical part of the system’s reliability not properly tested. No database performance, query optimization, and data consistency checks are performed, and problems in one of these areas could affect the entire system. Also, no external service is included among the tested components, while API validation and contract testing are crucial in a production environment.

- **Strength 1: Comprehensive testing coverage**

All the main testing techniques required to properly test the components used in the application are employed. Functional testing ensures that all the requirements are met, performance testing verifies the non-functional requirements and identifies any inefficiency, usability testing checks the user interaction, and finally load and stress testing evaluate the system’s behavior under specific volumes and conditions of usage. Operating in this way increases the probability of identifying problems, and makes it more certain that requirements are met.